BioTest: Metamorphic Testing for Textual VCF and SAM Parsers

Anonymous Authors
Anonymous Institution
anonymous@example.org

***Abstract*—**
Bioinformatics file-format parsers determine whether a variant call, alignment flag, or sample annotation reaches downstream analysis intact. Pipeline-level benchmarking efforts [1] assume parsers are sound, yet the textual VCF and SAM specifications leave many record-level semantics to de facto consensus, so testers operate without an external oracle [2]. We present BIOTEST, a metamorphic testing pipeline that replaces the human MR-identification step, the dominant cost in metamorphic-testing adoption [3], with a large language model (`deepseek-v4-flash`) grounded in the published specifications via retrieval-augmented prompting. The LLM mines identity-preserving relations once per format. Inputs flow through a thirteen-rank corpus stack and are graded by multi-runner consensus across $N \geq 3$ independent implementations. The per-SUT user cost is the parse-and-emit shim that mainstream coverage-guided fuzzers already require, with a small canonicalization extension that emits canonical JSON. Across six $(\mathrm{SUT}, \mathrm{format})$ configurations spanning four parser languages, BIOTEST attains the highest line coverage of any tool we measured on five of six SUTs (every VCF, plus htsjdk/SAM and biopython/SAM) and trails libFuzzer by 4.75 pp only on seqan3/SAM. On mutation adequacy it matches Atheris on vcfpy/VCF (parity within sample standard deviation), sits within a 0.4–4.2 pp deficit on three close-margin configurations, exceeds libFuzzer on seqan3/SAM, and trails Atheris by 32.5 pp on byte-content-lenient biopython/SAM. On a real-bug benchmark of 32 historical parser bugs [4] it detects 10, spanning htsjdk (Java), vcfpy (Python), and noodles (Rust), where every crash-only input fuzzer in the slate detects zero.

***Index Terms*—metamorphic testing, mutation testing, fuzz testing, bioinformatics, parser testing, oracle problem, software testing**

I. INTRODUCTION

A misread alignment flag, a 1-based coordinate parsed as 0-based, or a silent record drop in a Variant Call Format (VCF) file propagates from the parser to every downstream tool that consumes its output. Independent variant-calling pipelines disagree substantially on the same input data [5], and the discrepancy has motivated sustained investment in pipeline-level benchmark infrastructure such as Genome in a Bottle [6] and the GA4GH best-practice protocol [1]. These efforts grade the joint behaviour of caller and parser under the assumption that parsers are sound. Whether they are is rarely tested directly. Concrete consequences are easy to find in upstream issue trackers: htsjdk SAM regressions that accepted reference names with delimiters disallowed by SAM 1.6 and that over-strictly rejected reads with template lengths above 2^{29}, and a vcfpy regression that silently accepted symbolic ALT alleles VCF restricts to gVCF mode. Such regressions corrupt records that downstream callers treat as authoritative, and the error propagates to every downstream analysis. The textual specifications fix the on-disk grammar but leave many record-level semantics to de facto consensus, so testers operate without an external oracle [2].

Metamorphic testing replaces "what should the output be?" with relations on related outputs [3], [7]. File-format parsers fit cleanly: identity-preserving transforms over the spec-defined grammar yield equivalence relations derivable from the specification rather than from the parser source. The dominant cost is the relations themselves. The field's open-challenge survey [3] catalogues MR identification as the central problem, and existing studies report relations specified per SUT by human experts in the SUT's host language [8]. The bioinformatics ecosystem maintains parsers for VCF and SAM in four host languages simultaneously (htsjdk in Java, biopython and vcfpy in Python, noodles in Rust, seqan3 in C++), so a hand-authored per-SUT pipeline does not scale. The strongest competing approaches do not scale either: coverage-guided fuzzers require bytecode rewriting, sanitizer-instrumented builds, or fork-server harnessing on top of the parse-and-emit shim every fuzz tool needs.

We describe BIOTEST, a metamorphic testing pipeline that replaces the human MR-identification step with an LLM-driven, spec-grounded mining stage. The published specification is indexed for retrieval-augmented prompting [9]. An LLM (`deepseek-v4-flash`) proposes identity-preserving transforms grounded in the spec rule each transform claims to preserve, validated against a seed corpus before admission. Inputs flow through a thirteen-rank stack (Ranks 1–7 metamorphic, Ranks 8–13 augmentation), and the oracle is multi-runner consensus across $N \geq 3$ independent parsers. The user's per-SUT effort is a parse-and-emit shim that emits a canonical-JSON record set. BIOTEST adds no per-SUT MR, oracle predicate, seed-corpus curation, or instrumentation on top. Figure 1 gives the visual overview.

We evaluate BIOTEST on six $(\mathrm{SUT}, \mathrm{format})$ configurations against five coverage-guided fuzzers and a Pure-Random floor across three axes: end-of-run line coverage, mutation adequacy, and a real-bug benchmark [4] of 32 verified parser bugs (a tool detects a bug iff some input it produces triggers the buggy version but not the patched version). BIOTEST is competitive with the strongest coverage-guided fuzzer on text-rich parsers and trails on byte-content-lenient ones, consistent

with the empirical pattern of Klees et al. [10] and Böhme et al. [11]. On the bug benchmark it detects 10 of 32 parser bugs across three host languages (Java/htsjdk, Python/vcfpy, Rust/noodles), where every crash-only input fuzzer in the slate detects zero. Detection is end-to-end audited under a pre-fix / post-fix differential predicate in the ground-truth fuzzing style of Magma [4], on inputs the tool's adapter produced during the per-cell budget rather than on the manifest's canonical proof-of-vulnerability files alone.

a) Contributions.:

(i) BIOTEST, a metamorphic testing pipeline for textual VCF and SAM parsers in which an LLM mines spec-grounded MRs (rather than per-SUT hand authoring) under RAG retrieval and seed-corpus validation, with parser behaviour graded by a multi-runner consensus oracle.

(ii) A thirteen-rank corpus stack that combines grammar-driven metamorphic transforms with structural and lenient-byte augmentation, plus empirical characterisation of when the augmentation ranks fail.

(iii) A three-axis evaluation (line coverage, mutation adequacy, 32-bug real-bug benchmark) under matched protocols that maps where grammar-driven generation matches, leads, and trails coverage-guided fuzzing under the bounded per-SUT effort envelope (§V).

b) Roadmap.: §II surveys prior work in metamorphic testing, coverage-guided fuzzing, mutation testing, and bioinformatics parser correctness. §III and §IV present the design and implementation of BIOTEST. §V reports the three-axis evaluation, and §VI discusses threats to validity. §VII concludes.

II. RELATED WORK

BIOTEST sits at the intersection of metamorphic testing (supplying the oracle), coverage-guided greybox fuzzing (the strongest baselines), and mutation testing (the adequacy criterion). Our emphasis is on positioning rather than survey recapitulation.

A. Metamorphic Testing

Metamorphic testing was introduced by Chen et al. [7] as a partial answer to the oracle problem [2], [12]. Rather than asserting an absolute output, one asserts a metamorphic relation constraining how outputs change under structured input perturbations. Segura et al. [8] survey empirical adoption and Chen et al. [3] catalogue the open challenges, of which MR identification is the dominant cost in the technique's adoption.

Several lines of work attack MR identification by mining relations from existing artefacts. MeMo [13] extracts MRs from natural-language phrases in Javadoc comments, and MR-Scout [14] synthesises MRs from user-written test cases. BIOTEST mines from a different source, the published format *specification* itself. An LLM, grounded in the spec text via retrieval-augmented prompting and gated by a seed-corpus validator, proposes identity-preserving transforms that instantiate a parametric round-trip relation ($out(parse(x)) \equiv out(parse(T(x)))$). The novelty we claim is the combination of spec-grounded LLM-driven MR mining with a SUT-agnostic generator and consensus oracle, not any individual component.

B. Coverage-Guided Greybox Fuzzing

The in-process baselines we compare against typically find 2 to $5\times$ more bugs than blackbox baselines on a 24-hour budget [10], an advantage Böhme et al. [11] explain information-theoretically. Grammar-aware fuzzers (Zest [15], Nautilus [16], Superion [17], Smart Greybox Fuzzing [18], Fuzz4All [19]) recover structural fluency but require SUT-specific harness and build work that exceeds C2.

C. Mutation Testing

We use mutation testing as an adequacy proxy. Just et al. [20] establish that mutation kills correlate with real-fault detection at $r{\approx}0.7$ on Defects4J, and Andrews et al. [21] earlier showed mutants are an acceptable substitute for hand-seeded faults under standard operator sets. Papadakis et al. [22] survey the field and catalogue the equivalent-mutant problem. We adopt Vikram et al. [23] kill-feedback corpus selection at the pre-grading minimisation step. We complement the proxy with the real-bug benchmark of §V-C.

D. Bioinformatics Pipeline and Variant-Caller Testing

The bioinformatics community has built benchmarking infrastructure around *variant-caller correctness*. Genome in a Bottle [6] provides gold-standard variant calls, and the GA4GH best-practice protocol [1] (`hap.py`, `vcfeval`) controls for representational equivalence between a candidate VCF and the truth set. O'Rawe et al. [5] document substantial inter-pipeline disagreement on the same input data. These efforts grade the joint behaviour of caller, reference, and downstream parsing under the assumption that VCF and SAM parsers are sound. BIOTEST attacks the parser-correctness layer directly. A metamorphic violation, parser disagreement, or silent record drop that BIOTEST surfaces is a defect in a component that GIAB-style benchmarks assume to be sound, and the two efforts are therefore complementary.

III. DESIGN

BIOTEST treats a parser as an opaque function from bytes to a structured record set and exercises it via spec-derived metamorphic relations [3], [7], [8], multi-runner consensus oracles, and coverage-steered corpus augmentation. The framework remains useful across heterogeneous parser implementations of the same format without per-SUT engineering effort beyond the parse-and-emit shim that coverage-guided fuzzers already require. VCF [24] and SAM [25] are partially context-sensitive textual formats: header declarations constrain record-level types, and they admit multiple semantically-equivalent surface forms (tag order, integer encodings, mandatory-vs-default values) that make them amenable to metamorphic testing.

A. Design Constraints

BIOTEST is built on three constraints. **C1: SUT-agnostic core.** The framework contains no per-SUT logic in any phase. **C2: Per-SUT user effort bounded by what coverage-guided fuzzers already require.** Onboarding a parser is a thin *harness* that calls the parser and emits a canonical-JSON record set, plus a ∼50-line Python *runner* subclass that wraps the harness for the orchestrator. The harness conforms to a fixed per-format canonical schema (typed fields, deterministic ordering, 1-based coordinates). BIOTEST adds *no other per-SUT artefact*: no metamorphic relation, oracle predicate, seed corpus, instrumented build, bytecode-rewriting agent, or fork-server harness. **C3: Semantic-digest oracle.** A violation is declared when the observable parse outcome (parse-success flag, canonical-JSON digest, exception class) diverges, with no per-SUT assertion code. The oracle's intermediate strength forfeits coverage-feedback's bug-finding advantage on byte-content-lenient parsers [10], [11] in exchange for portability across four parser languages.

B. Pipeline

The framework runs as five phases (Figure 1). Phases A and B run once per format, and phases C, D, and E run per iteration on the SUT pool. **Phase A** chunks the published specification and embeds the chunks into a vector store for retrieval-augmented prompting [9], parametric over format and spec version. **Phase B** prompts an LLM to propose candidate metamorphic relations $out(parse(x)) \equiv out(parse(T(x)))$, each grounded in spec rules retrieved from the Phase A store and validated against a reference parser on a seed corpus before admission. **Phase C** routes each input x and its transform $T(x)$ through every configured parser runner. Outputs are canonicalised to JSON and graded by the consensus oracle of §III-D, which layers metamorphic equivalence on top of differential agreement [26]. **Phase D** reduces coverage telemetry to per-class blind-spot tickets and folds the highest-yield uncovered classes back into Phase B's prompt. **Phase E** content-hashes inputs accepted by at least one runner and runs the augmentation operators of §III-C (Ranks 8–13).

C. Corpus Stack

A pure metamorphic test loop ceilings at modest line coverage on file-format parsers. We therefore widen the corpus along two tiers ordered by decreasing semantic preservation. Ranks 1–7 are semantics-preserving metamorphic transforms whose outputs the consensus oracle scores under the metamorphic predicate. Ranks 8–13 trade strict semantic equivalence for branch and value diversity, and their outputs feed mutation grading and the differential-only oracle, which does not require metamorphic validity to declare a kill.

- **Ranks 1–7 (metamorphic).** LLM-synthesised seeds (R1), upstream-derived corpora (R2), spec-rule-targeted malformed mutators (R3), Hypothesis-targeted property-based generation (R4), API-query relations $P(parse(x)) \equiv P(parse(T(x)))$ over reflection-discovered scalar getters (R5), opt-in LLM-synthesised relations (R6), and theme-restricted spec-invariant registries (R7).
- **R8 corpus keeper.** Persists every transformed input accepted by at least one runner, deduplicated by canonical AST-hash.
- **R9 value diversifier.** Perturbs spec-bounded numerical fields (POS, QUAL, MAPQ, AF, DP, TLEN) within bounds, gated by a canonical-normaliser validity check.
- **R10 byte fuzzer.** Validity-gated bit-flip, byte-substitution, byte-insert, and byte-delete operators on non-header positions.
- **R11 boundary-value generator.** Boundary Value Analysis [12] on spec-bounded numerical fields.
- **R12 structural variant generator.** Spec-derived enumerator emitting branch-diverse inputs (CIGAR-shape mixes, tag-type permutations, multi-sample records, varied FILTER/INFO/FORMAT compositions).
- **R13 lenient byte fuzzer.** Weak-gate byte-level mutator retaining files with at least one line. Not metamorphic, consumed only by mutation-adequacy grading.

D. Consensus Oracle

Phase C parses each input with $N \geq 3$ SUTs drawn from the runner pool. The VCF pool is htsjdk, vcfpy, noodles, and pysam. The SAM pool is htsjdk, biopython, seqan3, and pysam. When the SUT under grading is one of these runners, the remaining three voters supply the consensus. Each runner emits a parsed record set canonicalised to JSON and hashed, alongside an exception class on failure. The oracle classifies the input under three layered predicates. (i) Differential consensus: if a strict majority of runners agrees on a digest $\sigma^\star$ and runner i disagrees, runner i is implicated. (ii) Metamorphic consensus: if $\sigma^\star_x = \sigma^\star_{T(x)}$ but $\sigma^x_i \neq \sigma^{T(x)}_i$, runner i is implicated. (iii) Relation quarantine: if the consensus disagrees on x versus $T(x)$, the relation has misclassified what the spec says is semantics-preserving. The choice $N \geq 3$ is the minimum that admits a strict majority and distinguishes "one runner is wrong" from "the spec is ambiguous" [2].

E. Cross-Tool Fairness Recipe

§V grades BIOTEST against coverage-guided fuzzers under four identical conditions. Each pair uses the same mutation engine (PIT [27] for htsjdk, mutmut [28] for vcfpy and biopython, cargo-mutants [29] for noodles, a DIY mull-style AOR/ROR/LOR operator set [20], [30] for seqan3), the same target-class scope, the same kill predicate (parse-success flip, canonical-JSON divergence, or crash-class flip) scored as $killed/reachable$ [22], [27], and the same wall-clock budget per pair across n reps. Only the corpus differs.

Wall-clock parity is not iteration-cost parity. In-process fuzzers iterate faster than fork-server fuzzers, and the imbalance shows up on seqan3/SAM as a 17 pp gap between AFL++ [31] and libFuzzer [32] despite identical mutants. Every corpus is trimmed to ≤ 200 files by an AFL-cmin-style minimiser using per-file kill-cardinality on Python SUTs [23]

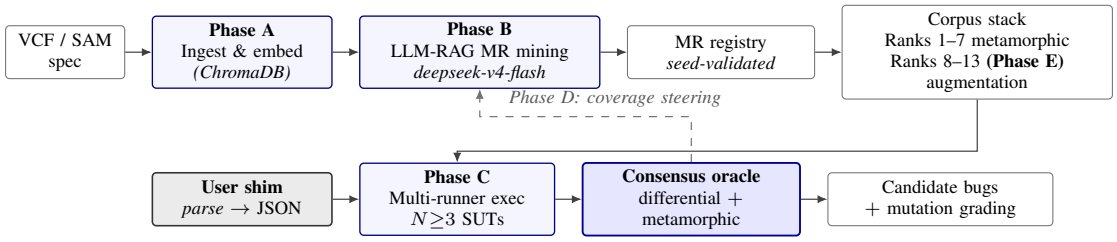

Fig. 1. BIOTEST pipeline. Phases A and B run once per format, and phases C, D, and E run per iteration. The dashed arrow marks Phase D coverage feedback into Phase B. The user's only per-SUT artefact is the parse-and-emit shim, the same artefact in-process coverage-guided fuzzers require.

and an SUT-agnostic outcome-fingerprint elsewhere. The selector adds $+1$ to $+2$ pp on pairs where it actively trims. As an uninformed-fuzzing reference [10] we add a Pure-Random byte emitter under identical conditions and report it as a per-pair floor.

IV. IMPLEMENTATION

BIOTEST is a Python orchestrator. Per-language SUT runners delegate parser invocation to the SUT's native toolchain, and the five phases of §III are realised concretely as follows.

a) Specification ingest (Phase A).: The published format specification is segmented at rule and field-definition boundaries, embedded with a sentence-transformer encoder, and indexed in a ChromaDB vector store. The same store serves every SUT for the format and is rebuilt only when the spec version changes.

b) LLM mining (Phase B).: The runs reported here use `deepseek-v4-flash` via the DeepSeek API. The orchestrator is provider-agnostic. The prompt retrieves the top-ranked spec chunks by cosine similarity to the rule the proposal must cite [9] and constrains the model to a whitelist of $\sim$ 20 atomic transforms registered in the framework. Each candidate MR traverses three sequential gates. The whitelist gate rejects unrecognised transforms. The hash gate collapses candidates whose $(format, scope, transform\_steps)$ tuple matches an already-admitted MR, so LLM nondeterminism does not inflate the registry. The seed-corpus gate validates the round-trip equality on a hand-curated reference corpus before admission, and is the load-bearing rejection cause across recorded snapshots. The first targeted mining run on the `ordering_invariance` intent for VCF admitted three enforced MRs out of seven candidate themes, and the per-pair enforced count in the headline runs ranges from 3 for SAM to 10 for VCF. Figure 2 traces the pipeline through one such accepted MR with real artefacts from the spec chunk to the validation outcome.

c) Runner architecture (Phases C, D).: The runner subprocesses the SUT's harness, parses the canonical-JSON digest, and exposes the result through a uniform interface. Java harnesses run on a per-test JVM, Python harnesses import the parser directly, Rust harnesses run via `cargo test` against a generated crate, and C++ harnesses are compiled once as a gcov-instrumented binary. Coverage is collected with JaCoCo for Java, `coverage.py` for Python, `cargo-llvm-cov` for Rust, and `gcovr` for C++.

d) Augmenters (Phase E).: Rank 12 and Rank 13 ship as stand-alone generators that are not LLM-mined. Rank 12 outputs feed both the metamorphic oracle and mutation grading. Rank 13 outputs feed only the latter, since Rank 13 is intentionally not metamorphic.

e) Reproducibility.: The orchestrator, SUT source pins, mutation engines, coverage tools, and baseline fuzzers ship as a single Docker image on Ubuntu 22.04. The C++ mutation engine is a re-implemented default-operator set rather than upstream Mull [30] due to a glibc/Ubuntu version conflict. The artefact is deposited as an anonymised Zenodo record (DOI placeholder for double-blind review, de-anonymised at camera-ready) containing the Docker image digest, the source-tree snapshot, the MR-curation log, and the per-pair raw kill counts and bug-bench records. We commit to the venue's artefact-evaluation track.

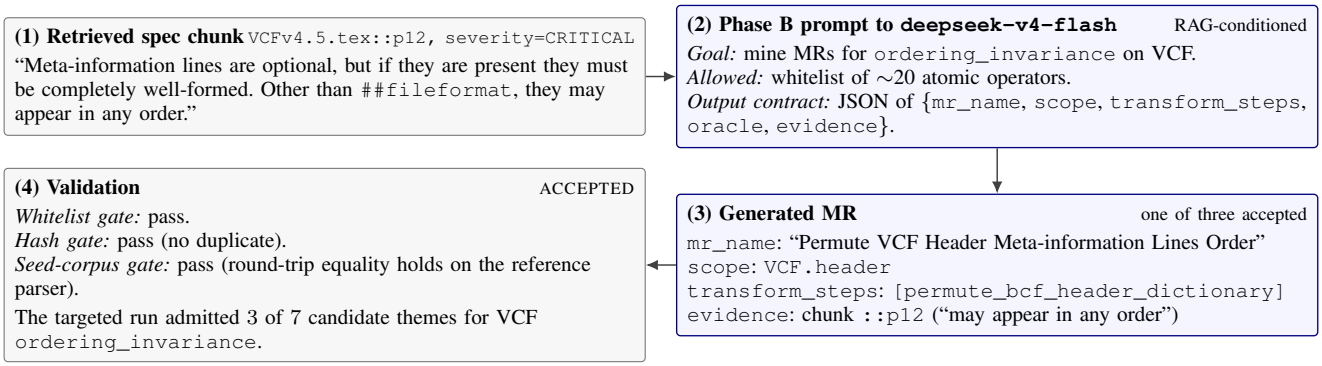

Fig. 2. LLM I/O traced through one accepted metamorphic relation. Panels 2 and 3 are LLM-touched (blue), panels 1 and 4 are framework data (grey). Every panel reproduces an artefact committed in the BioTest source tree.

V. Evaluation

We evaluate BioTest along three complementary axes. §V-A reports final line coverage of the corpus produced by BioTest relative to corpora produced by mainstream coverage-guided fuzzers under the matched-engine fairness recipe of §III-E. §V-B grades the same corpus on mutation adequacy under identical mutation engines and target-class filters. §V-C reports a real-bug benchmark of 32 verified historical parser bugs. Mutation kills correlate with real-fault detection at $r{\approx}0.7$ [20], so the three axes together provide construct triangulation.

The evaluation spans six $(\mathrm{SUT}, \mathrm{format})$ pairs: htsjdk/VCF, htsjdk/SAM (Java), vcfpy/VCF, biopython/SAM (Python), noodles/VCF (Rust), and seqan3/SAM (C++). All runs execute inside the reproducibility container described in §IV.

A. Code Coverage

a) Setup.: We measure line coverage on six $(\mathrm{SUT}, \mathrm{format})$ pairs, scoped by the same target-class filter that the mutation engine of §III-E uses so coverage and the mutation-adequacy metric of §V-B are graded on the same SUT surface. Line coverage (the fraction of source lines in each parser executed at least once during a tool's campaign) is collected by each language's standard coverage instrumentation (specific backends are listed in §IV). We report final-attainment line coverage, the end-of-campaign value each tool reaches inside its scope, because BioTest's Phase-D feedback loop (Figure 1) terminates on its own stop-criteria rather than at a fixed wall-clock tick. Within each row of Table I the scope filter and instrumentation backend are identical across columns. Line percentages are directly comparable as final attainment.

b) Headline observations and take-away.: BioTest attains the highest mean final line coverage of any tool we measured on five of six SUTs: $45.70{\pm}1.70\%$ on htsjdk/VCF, $74.12{\pm}1.52\%$ on vcfpy/VCF, $37.10{\pm}0.90\%$ on noodles/VCF, $30.90{\pm}0.10\%$ on htsjdk/SAM (a 5.85 pp lead over Jazzer), and $54.30{\pm}0.00\%$ on biopython/SAM (a 0.32 pp lead over Atheris). On seqan3/SAM BioTest trails libFuzzer by 4.75 pp (93.70% vs 98.45%). The result has a concrete structural explanation. Five of the six parsers expose declarative validators (`INFO`, `FORMAT`, contig declarations on VCF; CIGAR-shape, tag-type, and flag-bit checks on the SAM parsers) that BioTest's spec-grounded generator exercises preferentially because each is named in the spec and retrieved by LLM mining. Only seqan3/SAM retains the pattern Klees et al. and Böhme et al. [10], [11] report on byte-content-lenient parsers, where coverage-guided byte mutators accumulate inputs reaching rare control-flow paths more efficiently than spec-driven enumeration.

B. Mutation Adequacy

a) Setup.: Six $(\mathrm{SUT}, \mathrm{format})$ pairs span four parser languages and four mutation engines (PIT for htsjdk, mutmut for vcfpy and biopython, cargo-mutants for noodles, a DIY mull-style operator set for seqan3) under the fairness recipe of §III-E. Close-margin pairs run at $n_{\mathrm{BT}}{=}10$ with achieved baseline n of $10/6/9/6$ for htsjdk/VCF, htsjdk/SAM, vcfpy/VCF, noodles/VCF. Biopython and seqan3 retain $n{=}4$ where samples are deterministic ($\sigma{=}0.00$) or operate at budget-termination. For biopython, Atheris terminates at the 262nd of 523 generated mutants and we cap BioTest at the same 262 (full-population result $25.81{\pm}0.00\%$). Baselines and BioTest share the Tier-1+2 spec-example seed foundation. BioTest's corpus also carries its Phase-D synthetic outputs and Rank-12/13 augmentation seeds because these are part of BioTest's deployed pipeline, so mutation deltas reflect both generation strategy and seed pool on each side.

b) Significance and per-pair results.: We compute exact two-sided Mann–Whitney U by full enumeration of rank arrangements, Vargha–Delaney $\hat{A}_{12}$ [33], and Holm–Bonferroni correction at $k{=}4$. The deficit on htsjdk/VCF, htsjdk/SAM, and noodles/VCF ranges from -0.35 pp (noodles) through -2.47 pp (htsjdk/SAM) to -4.19 pp (htsjdk/VCF), with $\hat{A}_{12} \in \{0.000, 0.111, 0.183\}$ in the "large" magnitude band. All three clear family-wise $\alpha{=}0.05$ under Holm–Bonferroni (strongest signal htsjdk/VCF, Holm $p{=}4.0 \times 10^{-5}$). On vcfpy/VCF the gap is -0.05 pp ($87.32{\pm}0.28$ vs. Atheris $87.37{\pm}1.83$, $\hat{A}_{12}{\approx}0.5$), inside both samples' standard deviations, and we claim parity [34]. We do not claim "match" on the three deficit

TABLE I

FINAL LINE COVERAGE MEAN ± SAMPLE STD (%) ACROSS ALL (SUT, FORMAT) PAIRS AND TESTING TOOLS. EACH COLUMN'S TOOLS SHARE THE SAME SCOPE FILTER AND LANGUAGE-NATIVE COVERAGE TOOL. "—" INDICATES THE TOOL IS NOT LANGUAGE-APPLICABLE.

	VCF			**SAM**		
Tool	htsjdk	vcfpy	noodles	htsjdk	biopython	seqan3
Pure-Random	1.49±0.05	2.72±0.00	0.00±0.00	2.19±0.26	1.84±1.16	78.30±8.07
Jazzer [35]	34.69±0.03	—	—	25.05±0.18	—	—
Atheris [36]	—	55.01±0.04	—	—	53.98±0.48	—
cargo-fuzz [37]	—	—	22.86±0.11	—	—	—
libFuzzer [32]	—	—	—	—	—	**98.45**±0.58
AFL++ [31]	—	—	—	—	—	79.11±0.00
BIOTEST	**45.70**±1.70	**74.12**±1.52	**37.10**±0.90	**30.90**±0.10	**54.30**±0.00	93.70±0.30

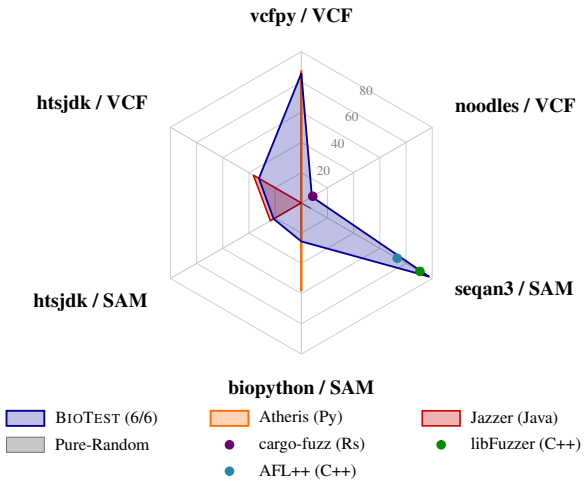

Fig. 3. Mutation kill rate per (SUT, format), means over n replicates per tool (n in §V-B). Background: concentric hexagons at the 20/40/60/80/100% levels with six radial axes. Tools spanning multiple SUTs (BIOTEST, Atheris, Jazzer, Pure-Random) are drawn as filled shaded polygons; single-SUT tools (cargo-fuzz, libFuzzer, AFL++) are drawn as a coloured dot at the value position on their applicable axis.

pairs: Klees et al. [10] report a 2–5× advantage for coverage-guided fuzzers, and we measure how close grammar-driven generation comes under our design constraints.

c) Biopython: coverage-guided fuzzing genuinely wins.: The −32.5 pp deficit on biopython is the largest signal in the radar, measured under the matched-budget protocol that caps both BIOTEST and Atheris at the first 262 generated mutants. Biopython's parser applies a strict structural validator but accepts byte-level variation within field values, and Atheris accumulates files that pass the validator and trip rare error paths. Enabling Rank-12 structural and Rank-13 lenient-byte-fuzz augmentation does not move the mean (25.48±0.79 vs. un-augmented 25.57±0.00, $\hat{A}_{12}$=0.50). This is the pattern Klees et al. and Böhme et al. [10], [11] report on byte-content-lenient parsers. Closing the gap requires per-SUT bytecode instrumentation, outside our C2 envelope.

d) seqan3: corpus retention matters.: On seqan3/SAM BIOTEST exceeds libFuzzer [32] by +6.87 pp, with both samples deterministic (σ=0.00). The mechanism is corpus retention under identical engine-side kill semantics. BIOTEST's spec-driven generator constructs inputs reaching distinct typed-exception classes (CIGAR-shape, tag-type, flag-bit boundaries via Rank 12), whereas libFuzzer's coverage-guided byte mutator retains crashes and discards typed-exception inputs. Mutations flipping one typed-exception class to another register as kills only when the corpus contains an input reaching the original class [2].

Construct- and internal-validity threats are detailed in §VI.

C. Real-Bug Detection

a) Setup and manifest construction.: We complement §V-B with a real-bug benchmark in the Magma [4] / FuzzBench [40] ground-truth style. The manifest is 32 historical parser bugs (23 VCF +9 SAM), each pinned to a verified $V_{\text{pre}} \rightarrow V_{\text{post}}$ commit pair. Candidates are sourced from each SUT's upstream release notes and `bug`-label PRs over the last five years (we do not mine `git log` for suspicious commits). A candidate enters iff all four hold: (i) reachable through the file-input pipeline as a parse exception, canonical-JSON value diff, or write-roundtrip diff; (ii) pre-fix and post-fix observably differ on the same input; (iii) the anchor pair reproduces empirically; (iv) both versions install as a pinned artefact. Failures are transparently dropped per standard ground-truth bench protocol [4], with the drop log in the artefact. Each entry ships a minimum-shape text PoV authored by reading the fix's `---/+++` diff.

b) Detection predicate, tool slate, and budget.: Detection follows a pre-fix / post-fix differential predicate in the ground-truth fuzzing style of Magma [4]: tool T detects bug B iff some input I that T's adapter produced during the budget satisfies $signal_T(I, V_{\text{pre}}) \neq signal_T(I, V_{\text{post}})$. The slate runs eight tools: the five coverage-guided fuzzers of §V-B, BIOTEST, an uninformed Pure-Random floor [10], and two unit-level Java baselines (EvoSuite [38] under an "EvoSuite-anchor" protocol that generates a JUnit suite against V_{post} and replays against V_{pre}, plus Randoop [39]). Per Magma-style methodology [4], every input fuzzer's seed corpus includes the manifest's canonical PoV alongside Tier-1+2 spec-example

TABLE II

REAL-BUG DETECTION PER (SUT, FORMAT) ON THE 32-BUG MANIFEST (23 VCF + 9 SAM; §V-C). EACH CELL IS THE NUMBER OF BUGS THE TOOL'S ADAPTER DETECTED UNDER THE PRE-FIX / POST-FIX DIFFERENTIAL PREDICATE, IN THE GROUND-TRUTH FUZZING STYLE OF MAGMA [4]. COLUMN HEADERS SHOW THE NUMBER OF MANIFEST BUGS IN EACH (SUT, FORMAT). "—" INDICATES THE TOOL IS NOT LANGUAGE-APPLICABLE TO THE SUT.

	htsjdk		**vcfpy**	**noodles**	**seqan3**[†]	
Tool	VCF (9)	SAM (3)	VCF (5)	VCF (9)	SAM (6)	Total (32)
Pure-Random [10]	0	0	0	0	0	0
Jazzer [35]	0	0	—	—	—	0
Atheris [36]	—	—	0	—	—	0
cargo-fuzz [37]	—	—	—	0	—	0
libFuzzer [32]	—	—	—	—	0	0
EvoSuite-anchor[♭] [38]	**4**	0	—	—	—	4
Randoop[♭] [39]	1	**2**	—	—	—	3
BIOTEST	**4**	**2**	**3**	**1**	0	**10**

[†] The six seqan3/SAM manifest entries are paradigm-out for any file-input or unit-level fuzzer (alignment-internal score, zero-record writer, BGZF concurrency, FASTA, harness-build failures, oracle mismatches), so every tool scores 0 on this column by construction. biopython/SAM is omitted because the verified manifest contains no biopython/SAM entries. [♭] Unit-level Java baselines (method-level test generation, not file-input fuzzing). Both are Java-only and run only on htsjdk.

seeds, equalising starting positions. The per-bug budget is 7200 s, gated by a language-eligibility matrix.

c) Headline.: Table II reports per-(SUT, format) detection counts. BIOTEST detects $10/32$ (htsjdk/VCF 4, htsjdk/SAM 2, vcfpy/VCF 3, noodles/VCF 1), and every other input fuzzer detects $0/32$. Forward (pre-fix rejects, post-fix accepts) fires on eight *over-strict* regressions spanning htsjdk stringency edge-cases, vcfpy lazy-evaluation traps, and a noodles writer round-trip. Reverse (pre-fix wrongly accepts, post-fix rejects) fires on two htsjdk *accept-when-should-reject* regressions where the buggy version silently mis-classified or mis-counted a record. Crash-only oracles surface neither shape without a deliberate exception [10], which is why the crash-only slate scores 0 on every (SUT, format) row.

d) Why the unit-level baselines reach four htsjdk bugs the input fuzzers do not.: EvoSuite [38] synthesises JUnit cases by genetic search over method-call sequences, and Randoop [39] constructs feedback-directed random sequences over the same surface. Both bypass file parsing entirely by constructing `VariantContext` / `SAMRecord` instances via the API. Of the four manifest entries reached this way and not by the input-fuzzing slate, one is a `VariantContextBuilder` regression structurally out-of-scope for any file-input fuzzer, and the other three are parse-time-reachable in principle but BIOTEST's spec-grounded MR transforms in the per-cell budget did not synthesise a trigger that isolated the patch from co-occurring spec violations.

e) Why BIOTEST *leads in absolute count.:* Despite seeing the SUT only through file parse + round-trip, BIOTEST is above both unit-level baselines on absolute count ($10 > 4 > 3$). Two effects dominate. First, BIOTEST applies to all four host languages while EvoSuite and Randoop are Java-only and run on the htsjdk row alone (12 of 32 cells). On that shared subset BIOTEST still leads ($6/12$ vs $4/12$ vs $3/12$). Second, the differential and metamorphic oracles surface silent value-disagreement bugs (e.g. mis-counted AC/AN/AF) that neither crash-only fuzzers nor unit-level GA can detect without an external comparator: crash-only oracles by construction [10], and unit-level GA because its assertions are derived from a single SUT version's observed output.

VI. THREATS TO VALIDITY

a) Construct.: Mutation kills are an imperfect proxy for real-fault detection. Equivalent mutants depress every tool's score equally, and we minimise the asymmetry by reusing standard operator sets [22]. The real-bug benchmark of §V-C grounds adequacy in real bugs as construct triangulation.

b) Internal: bug-bench manifest is post-audit.: The 32-bug manifest reflects a second-pass audit that dropped four SAM entries failing review and added three fresh htsjdk-SAM regressions, with a strict-stringency verification policy added after the initial freeze. Neither choice was blinded to which bugs BIOTEST's paradigm could reach. We apply the post-revision predicate uniformly to every tool to mitigate.

c) Internal: bug-detection attribution.: The bench seed corpus follows Magma-style PoV-seeding [4], equalising starting positions across all input fuzzers. Detection is verified on tool-output triggers under a pre-fix / post-fix differential predicate in the ground-truth fuzzing style of Magma [4]. Canary-based attribution from the same paper or magic-number injection per LAVA [41] would supersede the predicate but require per-bug source instrumentation across all four host languages, which we defer to future work.

d) Internal: rank-level ablation and LLM nondeterminism.: The thirteen-rank corpus stack ships in its deployed configuration with a single end-to-end kill rate per pair. The only rank-level ablation reported is Ranks 12-13 on biopython/SAM. Phase B uses a hosted LLM whose sampling is not bit-deterministic, and the seed-corpus MR validator is not blinded to downstream performance. We ship the curation

log with the artefact as mitigation. The four close-margin mutation-adequacy pairs run at $n{=}10$ and clear family-wise $\alpha{=}0.05$ under Holm–Bonferroni. Biopython and seqan3 remain at $n{=}4$ where samples are deterministic.

e) Internal: ablations not run.: Iteration-count parity on seqan3/SAM would isolate the wall-clock-vs-iteration confound behind the AFL++/libFuzzer gap. A Ranks 1-7 vs 1-13 ablation across every pair would quantify the augmentation-tier contribution beyond biopython/SAM. A hand-rolled differential-only oracle would isolate the $N{\geq}3$ majority-vote contribution.

f) External: bounded generalisation.: The evaluation covers six $(\mathrm{SUT}, \mathrm{format})$ pairs, two textual formats, and four parser languages. The result does not extend to checksum-validated binary formats (BCF, BAM, CRAM) where structural mutations are rejected at I/O time, nor to non-parser SUTs where the round-trip metamorphic relation does not apply.

VII. Conclusion

BioTest is a metamorphic-testing pipeline for textual VCF and SAM parsers whose per-SUT user effort is bounded by what coverage-guided fuzzers already require: a thin parse-and-emit shim and nothing else. The framework combines spec-grounded LLM-driven MR mining, a SUT-agnostic grammar-conforming generator, and a multi-runner consensus oracle applied uniformly across Java, Python, Rust, and C++. We evaluate it under a budget-and-mutation-engine parity protocol against five specialised fuzzers, a Pure-Random floor, and a real-bug benchmark of 32 verified historical parser bugs (§V).

Across the three axes the result is a clean trade-off. BioTest attains the highest line coverage of any tool we measured on five of six SUTs (every VCF, plus htsjdk/SAM and biopython/SAM) and trails libFuzzer by 4.75 pp only on seqan3/SAM. On mutation adequacy it matches Atheris on vcfpy/VCF (parity within sample standard deviation under the $n{=}10$ replication), sits within a 0.4–4.2 pp deficit of the strongest coverage-guided baseline on the three remaining close-margin configurations, trails by the margin Klees et al. and Böhme et al. [10], [11] predict qualitatively on byte-content-lenient biopython, and exceeds the strongest baseline on seqan3/SAM through corpus-side retention of typed-exception inputs. On the real-bug benchmark the gap is structurally explained by the dominance of differential-disagreement signals in the manifest's parse-time bug shapes. A natural future direction is a per-SUT coverage-feedback extension on top of the SUT-agnostic core, which would recover some of the biopython gap while preserving the deployment story for users who cannot pay the engineering cost.

References

[1] P. Krusche, L. Trigg, P. C. Boutros, C. E. Mason, F. M. De La Vega, B. L. Moore, M. Gonzalez-Porta, M. A. Eberle, Z. Tezak, S. Lababidi, R. Truty, G. Asimenos, B. Funke, M. Fleharty, B. A. Chapman, M. Salit, J. M. Zook, and Global Alliance for Genomics and Health Benchmarking Team, "Best practices for benchmarking germline small-variant calls in human genomes," *Nature Biotechnology*, vol. 37, no. 5, pp. 555–560, 2019.

[2] E. T. Barr, M. Harman, P. McMinn, M. Shahbaz, and S. Yoo, "The oracle problem in software testing: A survey," *IEEE Transactions on Software Engineering*, vol. 41, no. 5, pp. 507–525, 2015.

[3] T. Y. Chen, F.-C. Kuo, H. Liu, P.-L. Poon, D. Towey, T. H. Tse, and Z. Q. Zhou, "Metamorphic testing: A review of challenges and opportunities," *ACM Computing Surveys*, vol. 51, no. 1, pp. 4:1–4:27, 2018.

[4] A. Hazimeh, A. Herrera, and M. Payer, "Magma: A ground-truth fuzzing benchmark," *Proceedings of the ACM on Measurement and Analysis of Computing Systems (POMACS)*, vol. 4, no. 3, pp. 49:1–49:29, 2020.

[5] J. O'Rawe, T. Jiang, G. Sun, Y. Wu, W. Wang, J. Hu, P. Bodily, L. Tian, H. Hakonarson, W. E. Johnson, Z. Wei, K. Wang, and G. J. Lyon, "Low concordance of multiple variant-calling pipelines: practical implications for exome and genome sequencing," *Genome Medicine*, vol. 5, no. 3, p. 28, 2013.

[6] J. M. Zook, J. McDaniel, N. D. Olson, J. Wagner, H. Parikh, H. Heaton, S. A. Irvine, L. Trigg, R. Truty, C. Y. McLean, F. M. De La Vega, C. Xiao, S. Sherry, and M. Salit, "An open resource for accurately benchmarking small variant and reference calls," *Nature Biotechnology*, vol. 37, no. 5, pp. 561–566, 2019.

[7] T. Y. Chen, S. C. Cheung, and S.-M. Yiu, "Metamorphic testing: A new approach for generating next test cases," Department of Computer Science, Hong Kong University of Science and Technology, Tech. Rep. HKUST-CS98-01, 1998, republished as arXiv:2002.12543, 2020.

[8] S. Segura, G. Fraser, A. B. Sánchez, and A. Ruiz-Cortés, "A survey on metamorphic testing," *IEEE Transactions on Software Engineering*, vol. 42, no. 9, pp. 805–824, 2016.

[9] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems 33 (NeurIPS 2020)*, 2020, pp. 9459–9474.

[10] G. Klees, A. Ruef, B. Cooper, S. Wei, and M. Hicks, "Evaluating fuzz testing," in *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security (CCS 2018)*, 2018, pp. 2123–2138.

[11] M. Böhme, V. J. M. Manes, and S. K. Cha, "Boosting fuzzer efficiency: An information theoretic perspective," in *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE 2020)*, 2020, pp. 678–689.

[12] G. J. Myers, *The Art of Software Testing*. New York, NY: John Wiley & Sons, 1979.

[13] A. Blasi, A. Gorla, M. D. Ernst, M. Pezzè, and A. Carzaniga, "MeMo: Automatically identifying metamorphic relations in Javadoc comments for test automation," *Journal of Systems and Software*, vol. 181, p. 111041, 2021.

[14] C. Xu, V. Terragni, H. Zhu, J. Wu, and S.-C. Cheung, "MR-Scout: Automated synthesis of metamorphic relations from existing test cases," *ACM Transactions on Software Engineering and Methodology*, vol. 33, no. 6, 2024.

[15] R. Padhye, C. Lemieux, K. Sen, M. Papadakis, and Y. Le Traon, "Semantic fuzzing with zest," in *Proceedings of the 28th ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA 2019)*, 2019, pp. 329–340.

[16] C. Aschermann, T. Frassetto, T. Holz, P. Jauernig, A.-R. Sadeghi, and D. Teuchert, "NAUTILUS: Fishing for deep bugs with grammars," in *Proceedings of the 26th Annual Network and Distributed System Security Symposium (NDSS 2019)*, 2019.

[17] J. Wang, B. Chen, L. Wei, and Y. Liu, "Superion: Grammar-aware greybox fuzzing," in *Proceedings of the 41st International Conference on Software Engineering (ICSE 2019)*, 2019, pp. 724–735.

[18] V.-T. Pham, M. Böhme, A. E. Santosa, A. R. Căciulescu, and A. Roychoudhury, "Smart greybox fuzzing," *IEEE Transactions on Software Engineering*, vol. 47, no. 9, pp. 1980–1997, 2021.

[19] C. S. Xia, M. Paltenghi, J. L. Tian, M. Pradel, and L. Zhang, "Fuzz4All: Universal fuzzing with large language models," in *Proceedings of the 46th International Conference on Software Engineering (ICSE 2024)*, 2024, pp. 1–13.

[20] R. Just, D. Jalali, L. Inozemtseva, M. D. Ernst, R. Holmes, and G. Fraser, "Are mutants a valid substitute for real faults in software testing?" in

Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering (FSE 2014), 2014, pp. 654–665.

[21] J. H. Andrews, L. C. Briand, and Y. Labiche, "Is mutation an appropriate tool for testing experiments?" in *Proceedings of the 27th International Conference on Software Engineering (ICSE 2005)*, 2005, pp. 402–411.

[22] M. Papadakis, M. Kintis, J. Zhang, Y. Jia, Y. Le Traon, and M. Harman, "Mutation testing advances: An analysis and survey," *Advances in Computers*, vol. 112, pp. 275–378, 2019.

[23] V. Vikram, I. Laybourn, A. Li, N. Nair, K. O'Brien, R. Sanna, and R. Padhye, "Guiding greybox fuzzing with mutation testing," in *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA 2023)*, 2023, pp. 929–941.

[24] P. Danecek, A. Auton, G. Abecasis, C. A. Albers, E. Banks, M. A. DePristo, R. E. Handsaker, G. Lunter, G. T. Marth, S. T. Sherry, G. McVean, R. Durbin, and 1000 Genomes Project Analysis Group, "The variant call format and VCFtools," *Bioinformatics*, vol. 27, no. 15, pp. 2156–2158, 2011.

[25] H. Li, B. Handsaker, A. Wysoker, T. Fennell, J. Ruan, N. Homer, G. Marth, G. Abecasis, R. Durbin, and 1000 Genome Project Data Processing Subgroup, "The Sequence Alignment/Map format and SAMtools," *Bioinformatics*, vol. 25, no. 16, pp. 2078–2079, 2009.

[26] W. M. McKeeman, "Differential testing for software," *Digital Technical Journal*, vol. 10, no. 1, pp. 100–107, 1998.

[27] H. Coles, T. Laurent, C. Henard, M. Papadakis, and A. Ventresque, "PIT: A practical mutation testing tool for Java (tool demo)," in *Proceedings of the 25th International Symposium on Software Testing and Analysis (ISSTA 2016)*, 2016, pp. 449–452.

[28] A. Hovmöller, "mutmut — Python mutation tester," https://github.com/boxed/mutmut, 2024, accessed 2026-04-26; experiment used mutmut 2.5.1.

[29] M. Pool, "cargo-mutants — Rust mutation testing," https://github.com/sourcefrog/cargo-mutants, 2024, accessed 2026-04-26; experiment used cargo-mutants 27.0.

[30] A. Denisov and S. Pankevich, "Mull it over: Mutation testing based on LLVM," in *2018 IEEE International Conference on Software Testing, Verification and Validation Workshops (ICSTW)*, 2018, pp. 25–31.

[31] A. Fioraldi, D. Maier, H. Eißfeldt, and M. Heuse, "AFL++: Combining incremental steps of fuzzing research," in *Proceedings of the 14th USENIX Workshop on Offensive Technologies (WOOT 2020)*, 2020.

[32] LLVM Project, "libFuzzer — a library for coverage-guided fuzz testing," https://llvm.org/docs/LibFuzzer.html, 2024, accessed 2026-04-26; experiment used Clang 14 libFuzzer.

[33] A. Vargha and H. D. Delaney, "A critique and improvement of the CL common language effect size statistics of McGraw and Wong," *Journal of Educational and Behavioral Statistics*, vol. 25, no. 2, pp. 101–132, 2000.

[34] A. Arcuri and L. Briand, "A hitchhiker's guide to statistical tests for assessing randomized algorithms in software engineering," *Software Testing, Verification and Reliability*, vol. 24, no. 3, pp. 219–250, 2014, earlier version at ICSE 2011, DOI 10.1145/1985793.1985795.

[35] Code Intelligence GmbH, "Jazzer — coverage-guided, in-process fuzzer for the JVM," https://github.com/CodeIntelligenceTesting/jazzer, 2024, accessed 2026-04-26; experiment used Jazzer 0.22.x.

[36] Google LLC, "Atheris — a coverage-guided, native Python fuzzer," https://github.com/google/atheris, 2023, accessed 2026-04-26; experiment used Atheris 2.3.0.

[37] The Rust Fuzzing Authority, "cargo-fuzz — command-line tool for fuzzing with libfuzzer," https://github.com/rust-fuzz/cargo-fuzz, 2024, accessed 2026-04-26; experiment used cargo-fuzz 0.12.

[38] G. Fraser and A. Arcuri, "EvoSuite: Automatic test suite generation for object-oriented software," in *Proceedings of the 19th ACM SIGSOFT Symposium and the 13th European Conference on Foundations of Software Engineering (ESEC/FSE 2011)*, 2011, pp. 416–419.

[39] C. Pacheco, S. K. Lahiri, M. D. Ernst, and T. Ball, "Feedback-directed random test generation," in *Proceedings of the 29th International Conference on Software Engineering (ICSE 2007)*, 2007, pp. 75–84.

[40] J. Metzman, L. Szekeres, L. Simon, R. Sprabery, and A. Arya, "FuzzBench: An open fuzzer benchmarking platform and service," in *Proceedings of the 29th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE 2021)*, 2021, pp. 1393–1403.

[41] B. Dolan-Gavitt, P. Hulin, E. Kirda, T. Leek, A. Mambretti, W. Robertson, F. Ulrich, and R. Whelan, "LAVA: Large-scale automated vulnerability addition," in *Proceedings of the 37th IEEE Symposium on Security and Privacy (SP 2016)*, 2016, pp. 110–121.